UTL-X

Format-Agnostic Data Transformation

The Complete Guide

Ir. Marcel A. Grauwen

Creator of UTL-X

Version 1.0 — 2026

Table of Contents

1 Preface ... 15
- 1.1 Why This Book ... 15
- 1.2 Who This Book Is For ... 15
- 1.3 How to Read This Book ... 15
- 1.4 Conventions Used ... 15
- 1.5 Online Resources ... 15

2 Introduction ... 17
- 2.1 The Data Transformation Problem ... 17
- 2.2 What Is UTL-X? ... 17
- 2.3 The Swiss Army Knife Analogy ... 17
- 2.4 UTL-X vs The Alternatives ... 18
- 2.5 Architecture Overview ... 18
- 2.6 The UTL-X Ecosystem ... 19
- 2.7 A First Example ... 19
- 2.8 What's Ahead ... 20

3 Licensing: Open Source and Commercial ... 22
- 3.1 UTL-X Is Open Source (AGPL-3.0) ... 22
- 3.2 What You Can Do (Without Paying Anything) ... 22
- 3.3 The Key AGPL Condition ... 22
- 3.4 Your Transformations Are Yours ... 22
- 3.5 Usage Scenarios ... 23
- 3.6 The Subprocess Exception ... 23
- 3.7 The Commercial License ... 24
  - 3.7.1 Why It Exists ... 24
  - 3.7.2 Cloud Marketplace (Simplest Path) ... 24
  - 3.7.3 Direct Commercial License ... 24
- 3.8 Dual Licensing: How It Works ... 24
- 3.9 Frequently Asked Questions ... 24

4 Transformation: The Cornerstone of Integration ... 26
- 4.1 Why Transformation Matters ... 26
- 4.2 The Three Pillars of Integration ... 26
- 4.3 Transformation in ESB Architecture ... 26
- 4.4 Transformation in Workflow/BPM ... 26
- 4.5 Why UTLXe Needs Messaging Capabilities ... 26
- 4.6 The Canonical Data Model Pattern ... 26

5 Getting Started ... 27
- 5.1 Installation ... 27
  - 5.1.1 macOS (Homebrew) — Recommended ... 27
  - 5.1.2 Windows (Chocolatey) ... 27
  - 5.1.3 Linux / macOS / Windows — Direct Download ... 27
  - 5.1.4 Building from Source ... 28
  - 5.1.5 Native Binary vs JVM JAR ... 28
- 5.2 Your First Transformation ... 29
- 5.3 The .utlx File Structure ... 29
  - 5.3.1 Input Format Options ... 30

5.3.2 Output Format Options ... 30
5.3.3 Multi-Input ... 30
5.4 Trying It Out: Five Quick Examples ... 30
5.5 What's Next ... 31
6 UTL-X on the Command Line ... 32
6.1 Three Ways to Use the CLI ... 32
6.2 Expression Mode: The jq Experience ... 32
6.2.1 Raw Output (-r) ... 32
6.2.2 Expressions with Any Format ... 32
6.2.3 Complex Expressions ... 33
6.3 Identity Mode: The Flip ... 33
6.4 Shell Scripting Patterns ... 34
6.4.1 Batch File Conversion ... 34
6.4.2 Pipeline Integration ... 34
6.4.3 Using UTL-X in CI/CD ... 35
6.4.4 Combining with jq ... 35
6.5 The UTL-X jq Cheat Sheet ... 35
6.6 When to Use Which Mode ... 36
6.7 Performance Notes ... 36
7 The Three Executables: utlx, utlxd, and utlxe ... 37
7.1 One Language, Three Runtimes ... 37
7.2 utlx — The CLI (Command Line Interface) ... 37
7.3 utlxd — The Daemon (IDE Integration) ... 37
7.4 utlxe — The Engine (Production Runtime) ... 37
7.5 Comparison Table ... 37
7.6 Architecture: Shared Core, Different Shells ... 37
7.7 When to Use Which ... 37
7.8 The Wrapper Pattern ... 37
8 The UTL-X IDE ... 38
8.1 IDE Layout and Panels ... 38
8.2 Design-Time vs Runtime ... 38
8.3 Schema-to-Schema Mapping (Design-Time First) ... 38
8.4 Scaffolding ... 38
8.5 The UDM Tree Browser ... 38
8.6 The Function Library ... 38
8.7 The %utlx Header Editor ... 38
9 Language Fundamentals ... 39
9.1 Scalar Types ... 39
9.2 Objects ... 39
9.3 Arrays ... 39
9.4 The $input Variable ... 39
9.5 Variable Binding (let) ... 39
9.6 Conditional Expressions ... 39
9.7 Comments ... 39
9.8 Type Annotations (Optional) ... 39
10 The Universal Data Model (UDM) ... 40

10.1 Why UDM Exists . . . 40
10.2 UDM Types . . . 40
10.3 How Formats Map to UDM . . . 40
10.4 XML-Specific UDM Conventions . . . 40
10.5 UDM Navigation . . . 40
10.6 UDM and Type Coercion . . . 40
11 Schema-to-Schema Mapping . . . 41
11.1 Why Schema Mapping Matters . . . 41
11.2 How Traditional Tools Do It (and Why It's Broken) . . . 41
11.2.1 The XSLT Approach (Tibco BW, SAP CPI, Azure) . . . 41
11.2.2 The JSON Problem: Why JSON→XML→XSLT→XML→JSON Is Wrong . 41
11.2.3 The UTL-X Approach: Format-Agnostic Schema Mapping . . . 41
11.3 Tier 1: Instance Data Mapping . . . 41
11.3.1 Every Data Format Has an Accompanying Schema Format . . . 41
11.3.2 Instance-to-Instance Mapping Examples . . . 41
11.4 Tier 2: Schema-to-Schema (Metadata) Mapping . . . 41
11.4.1 Why Metadata Mapping Is Powerful . . . 41
11.4.2 All Possible Schema-to-Schema Combinations . . . 41
11.4.3 Why This Matters in Practice . . . 41
11.5 JSONATA: JSON Schema to JSON Schema Mapping . . . 41
11.6 Nested Schemas: Include and Import . . . 41
11.7 USDL: The Universal Schema Definition Language . . . 41
12 USDL: Universal Schema Definition Language . . . 42
12.1 Why USDL? . . . 42
12.2 USDL Syntax . . . 42
12.3 USDL Type System . . . 42
12.4 USDL Constraints . . . 42
12.5 USDL ↔ Other Schema Formats . . . 42
12.6 Using USDL in Transformations . . . 42
12.7 USDL as Documentation . . . 42
13 Format Support Overview . . . 43
13.1 Tier 1: Data Formats . . . 43
13.2 Tier 2: Schema Formats . . . 43
13.3 Format Detection . . . 43
13.4 Format Options . . . 43
13.5 Format Conversion Matrix . . . 43
14 Expressions and Operators . . . 45
14.1 Arithmetic Operators . . . 45
14.2 Comparison Operators . . . 45
14.3 Logical Operators . . . 45
14.4 String Operators . . . 45
14.5 Object Construction . . . 45
14.6 Array Construction . . . 45
14.7 Pipe Operator (|>) . . . 45
14.8 Nullish Coalescing and Safe Navigation . . . 45
15 Functions and Lambdas . . . 46

15.1 Lambda Expressions . . . 46
15.2 User-Defined Functions . . . 46
15.3 Higher-Order Functions . . . 46
15.4 Function Composition . . . 46
15.5 Closures . . . 46
15.6 The Standard Library (Overview) . . . 46
16 Standard Library . . . 47
16.1 String Functions (83 functions) . . . 47
16.2 Array Functions (67 functions) . . . 47
16.3 Math Functions (37 functions) . . . 47
16.4 Date/Time Functions (68 functions) . . . 47
16.5 Type Functions (27 functions) . . . 47
16.6 Encoding Functions (30 functions) . . . 47
16.7 Security Functions (16 functions) . . . 47
16.8 XML Functions (60 functions) . . . 47
16.9 Other Categories . . . 47
17 Security Library and Message Security . . . 48
17.1 The UTL-X Security Library (stdlib-security) . . . 48
17.2 Should Messages Be Encrypted in Transit? . . . 48
17.2.1 Transport-Level Encryption (mTLS) . . . 48
17.2.2 Message-Level Encryption (End-to-End) . . . 48
17.2.3 Why Encrypt-Transform-Re-encrypt Is a Maintenance Burden . . . 48
17.3 Digital Signatures and Integrity . . . 48
17.3.1 JCS (JSON Canonicalization Scheme) . . . 48
17.3.2 XML Digital Signatures (XMLDSig) . . . 48
17.3.3 JWS (JSON Web Signature) . . . 48
17.4 Future Security Functions (Vision) . . . 48
17.4.1 Encryption . . . 48
17.4.2 Signing . . . 48
17.4.3 Key Management Integration . . . 48
17.5 Large Files: CMIS and Object Storage References . . . 48
17.5.1 The Problem with Large Payloads . . . 48
17.5.2 The Claim Check Pattern . . . 48
17.5.3 CMIS (Content Management Interoperability Services) . . . 48
17.5.4 Avoid Base64 Embedding . . . 48
18 Pattern Matching and Control Flow . . . 49
18.1 Match Expressions . . . 49
18.2 Try/Catch . . . 49
18.3 Conditional Patterns . . . 49
18.4 Real-World Patterns . . . 49
19 Schema Validation . . . 50
19.1 Why Validate? . . . 50
19.2 Supported Schema Formats . . . 50
19.3 Pre-Validation and Post-Validation . . . 50
19.4 JSON Schema Validation . . . 50
19.5 XSD Validation . . . 50

19.6 Schema-Driven Transformations . . . 50
20 Pipeline Chaining and Multi-Hop Transformations . . . 51
20.1 What Is Pipeline Chaining? . . . 51
20.2 Why Pipelines? . . . 51
20.3 Single-Hop vs Multi-Hop . . . 51
20.3.1 Single-Hop (Simple Transformation) . . . 51
20.3.2 Multi-Hop (Pipeline) . . . 51
20.4 Multi-Hop with Additional Inputs per Stage . . . 51
20.4.1 The Problem . . . 51
20.4.2 The Solution: Additional Inputs per Hop . . . 51
20.4.3 When Is Multi-Hop with Additional Inputs Handy? . . . 51
20.5 Defining Pipelines . . . 51
20.5.1 Pipeline via UTLXe HTTP API . . . 51
20.5.2 Pipeline via CLI . . . 51
20.6 Multi-Input Transformations (Single Step) . . . 51
20.7 Pipeline Patterns . . . 51
20.7.1 Enrichment Pipeline . . . 51
20.7.2 Validation Sandwich . . . 51
20.7.3 Fan-Out . . . 51
20.7.4 Fan-In (Aggregation) . . . 51
20.7.5 Conditional Routing . . . 51
20.8 Performance Characteristics . . . 51
20.9 Status: Multi-Hop Additional Inputs . . . 51
21 XML Transformations . . . 53
21.1 XML in UTL-X . . . 53
21.2 Reading XML . . . 53
21.3 Writing XML . . . 53
21.4 XML Attributes in JSON/YAML Output . . . 53
21.5 Common XML Patterns . . . 53
21.6 XML Encoding . . . 53
21.7 XML Namespaces Deep Dive . . . 53
22 XML Attribute Handling: Design Decisions . . . 54
22.1 The Fundamental Problem . . . 54
22.2 How UTL-X Handles Attributes Internally (UDM) . . . 54
22.3 The _text Convention . . . 54
22.4 The writeAttributes Option . . . 54
22.5 What Gets Kept vs Dropped (Default) . . . 54
22.6 Why Drop by Default? (DataWeave-Compatible) . . . 54
22.7 JSON vs YAML Consistency . . . 54
22.8 Industry Conventions Reference . . . 54
22.9 FHIR XML: The value Attribute Convention . . . 54
23 JSON Transformations . . . 55
23.1 JSON in UTL-X . . . 55
23.2 Reading JSON . . . 55
23.3 Writing JSON . . . 55
23.4 JSON-Specific Functions . . . 55

23.5 Common JSON Patterns ... 55
23.6 JSON Lines (JSONL) ... 55
24 CSV Transformations ... 56
24.1 CSV in UTL-X ... 56
24.2 CSV Options ... 56
24.3 Reading CSV ... 56
24.4 Writing CSV ... 56
24.5 CSV-Specific Functions ... 56
24.6 Common CSV Patterns ... 56
24.7 Regional Formats ... 56
25 YAML Transformations ... 57
25.1 YAML in UTL-X ... 57
25.2 Reading YAML ... 57
25.3 Writing YAML ... 57
25.4 YAML Output Options ... 57
25.5 YAML-Specific Functions ... 57
25.6 Common YAML Patterns ... 57
25.7 YAML ↔ JSON Conversion ... 57
26 OData Transformations ... 58
26.1 What Is OData? ... 58
26.2 OData in UTL-X ... 58
26.3 Reading OData ... 58
26.4 Writing OData ... 58
26.5 OData-Specific Functions ... 58
26.6 Common OData Patterns ... 58
26.6.1 Dynamics 365 Integration ... 58
26.6.2 SharePoint Integration ... 58
26.6.3 SAP OData Integration ... 58
26.6.4 Power Platform ... 58
26.7 OData Schema (EDMX/CSDL) ... 58
26.8 OData ↔ Other Formats ... 58
27 Schema Formats ... 59
27.1 XSD (XML Schema Definition) ... 59
27.2 JSON Schema ... 59
27.3 Avro Schema ... 59
27.4 Protobuf (proto3) ... 59
27.5 OSCH (OData Schema / EDMX) ... 59
27.6 TSCH (Table Schema) ... 59
27.7 USDL (Universal Schema Definition Language) ... 59
28 XSD Design Patterns Deep Dive ... 60
28.1 The Eight XML Schema Design Patterns ... 60
28.1.1 1. Russian Doll ... 60
28.1.2 2. Salami Slice ... 60
28.1.3 3. Venetian Blind ... 60
28.1.4 4. Garden of Eden ... 60
28.1.5 5. Swiss Army Knife (Hybrid) ... 60

28.1.6 6-8. Chameleon, Abstract, Adapter ... 60
28.2 Pattern Detection in UTL-X ... 60
28.3 Pattern Conversion in UTL-X ... 60
28.4 XSD Patterns in Cross-Format Mapping ... 60
29 Cross-Format Patterns ... 61
29.1 The Power of Format Agnosticism ... 61
29.2 XML ↔ JSON Conversion ... 61
29.3 CSV ↔ JSON Conversion ... 61
29.4 XML → CSV (Flattening) ... 61
29.5 JSON → XML (Structuring) ... 61
29.6 Multi-Format Pipelines ... 61
29.7 Data Normalization Patterns ... 61
30 Enterprise Integration ... 63
30.1 UTL-X in Integration Architecture ... 63
30.2 Use Case: European E-Invoicing (Peppol/UBL) ... 63
30.3 Use Case: Healthcare (HL7 FHIR) ... 63
30.4 Use Case: Financial Services (SWIFT/ISO 20022) ... 63
30.5 Use Case: Retail/E-Commerce ... 63
30.6 Message Broker Integration ... 63
30.7 Error Handling in Production ... 63
31 Engine Lifecycle: Design-Time, Init-Time, Runtime ... 64
31.1 The Three Phases ... 64
31.1.1 Phase 1: Design-Time ... 64
31.1.2 Phase 2: Init-Time (Load / Compile) ... 64
31.1.3 Phase 3: Runtime (Execute) ... 64
31.2 Phase Diagram ... 64
31.3 How Strategies Affect Each Phase ... 64
31.4 Worker Threading Model ... 64
32 Cloud Deployment ... 65
32.1 UTLXe: The Production Engine ... 65
32.2 Azure Deployment ... 65
32.3 GCP Deployment ... 65
32.4 AWS Deployment ... 65
32.5 Docker and Kubernetes ... 65
32.6 Health and Monitoring ... 65
32.7 Hot Reload ... 65
33 SDKs and Wrapper Libraries ... 66
33.1 The Wrapper Pattern ... 66
33.2 Available SDKs ... 66
33.2.1 C# / .NET Wrapper (Built) ... 66
33.2.2 Go Wrapper (Built) ... 66
33.2.3 Python Wrapper (Planned) ... 66
33.2.4 Java / Kotlin Wrapper (Direct) ... 66
33.2.5 Node.js / TypeScript Wrapper (Planned) ... 66
33.3 Alternative: HTTP API (No SDK Needed) ... 66
33.4 Alternative: Dapr Sidecar (Cloud-Native) ... 66

33.5 SDK Comparison ... 66
34 Migration Guides ... 67
34.1 Migrating from XSLT ... 67
34.2 Migrating from DataWeave ... 67
34.3 Migrating from jq ... 67
34.4 Migrating from Custom Code ... 67
34.5 Migrating from Tibco BW / BWCE ... 67
35 Performance and Optimization ... 68
35.1 Execution Strategies ... 68
35.2 Performance Characteristics ... 68
35.3 Memory Optimization ... 68
35.4 GraalVM Native Image ... 68
35.5 Benchmarking ... 68
35.6 Production Tuning ... 68
36 Message Parsing, Memory, and Size Boundaries ... 69
36.1 How UTL-X Parses Messages ... 69
36.1.1 DOM-Style Parsing (What UTL-X Uses) ... 69
36.1.2 SAX-Style Parsing (Not Used by UTL-X) ... 69
36.1.3 StAX-Style Parsing (Not Used by UTL-X) ... 69
36.1.4 Summary: Parsing Approaches ... 69
36.2 Memory Expansion Factor ... 69
36.2.1 The Problem: Why 100KB of XML Becomes 5-10MB in Memory ... 69
36.2.2 Expansion Factors by Format ... 69
36.2.3 Real-World Examples ... 69
36.3 Practical Message Size Boundaries ... 69
36.3.1 Sweet Spot: 1 KB - 1 MB ... 69
36.3.2 Feasible: 1 MB - 10 MB ... 69
36.3.3 Not Recommended: 10 MB - 50 MB ... 69
36.3.4 Not Feasible: 50 MB+ ... 69
36.3.5 Size Boundary Summary ... 69
36.4 Memory Sizing Guide for Production ... 69
36.4.1 Formula ... 69
36.4.2 The 75% Rule ... 69
36.5 Future: Streaming Support? ... 69
36.5.1 Could UTL-X Support Streaming? ... 69
36.5.2 Split-Before-Transform Pattern ... 69
36.6 Comparison: UTL-X vs Streaming Tools ... 69
37 Logging, Observability, and Compliance ... 70
37.1 Logging in UTL-X ... 70
37.1.1 What UTL-X Logs ... 70
37.1.2 Log Levels ... 70
37.1.3 Configuring Logging ... 70
37.2 When NOT to Log: Compliance and Data Protection ... 70
37.2.1 PCI DSS (Payment Card Industry Data Security Standard) ... 70
37.2.2 GDPR (General Data Protection Regulation) ... 70
37.2.3 HIPAA (Health Insurance Portability and Accountability Act) ... 70

37.2.4 NEN 7510 (Dutch Healthcare Information Security) . . . 70
37.2.5 SOX (Sarbanes-Oxley Act — Financial) . . . 70
37.3 Designing for Compliance . . . 70
37.3.1 The Principle: Log Metadata, Not Data . . . 70
37.3.2 Audit Logging Pattern . . . 70
37.3.3 Masking in Error Messages . . . 70
37.4 Prometheus Metrics (Safe by Design) . . . 70
38 Quality Assurance and Testing . . . 71
38.1 UTL-X's Layered Testing Strategy . . . 71
38.2 Layer 1: Unit Tests (Kotlin) . . . 71
38.3 Layer 2: Format Tests (Kotlin) . . . 71
38.4 Layer 3: Module Tests (Kotlin) . . . 71
38.5 Layer 4: Integration Tests (Kotlin) . . . 71
38.6 Layer 5: Conformance Suite (Python) . . . 71
38.7 Auto-Capture: Recording Tests from CLI Usage . . . 71
38.8 UTLXe Engine Tests . . . 71
38.9 Test-Driven Transformation Development . . . 71
38.10 Continuous Integration . . . 71
39 Semantic Validation: Beyond Syntax . . . 73
39.1 Syntactical vs Semantic Validation . . . 73
39.1.1 What UTL-X Does Today: Syntactical Validation . . . 73
39.1.2 What UTL-X Does NOT Do: Semantic Validation . . . 73
39.1.3 The Gap . . . 73
39.2 Semantic Validation Languages . . . 73
39.2.1 Schematron (XML world) . . . 73
39.2.2 JSONTron . . . 73
39.2.3 FEEL (Friendly Enough Expression Language) . . . 73
39.2.4 Other Approaches . . . 73
39.3 How Semantic Validation Could Work in UTL-X . . . 73
39.3.1 Option 1: Built-in assertion syntax . . . 73
39.3.2 Option 2: Schematron support for XML output . . . 73
39.3.3 Option 3: FEEL integration for cross-format validation . . . 73
39.3.4 Option 4: Validation DSL (UTL-X native) . . . 73
39.3.5 Recommended Approach . . . 73
40 API Contracts and Data Contracts . . . 74
40.1 What Is an API Contract? . . . 74
40.2 Two Dimensions of Contracts . . . 74
40.2.1 Hierarchical Contracts (API-Centric) . . . 74
40.2.2 Relational Contracts (Data-Centric) . . . 74
40.2.3 Key Difference . . . 74
40.3 Hierarchical API Contract Standards . . . 74
40.3.1 OpenAPI (REST APIs) . . . 74
40.3.2 RAML (REST APIs — MuleSoft) . . . 74
40.3.3 SOAP / WSDL (XML Web Services) . . . 74
40.3.4 AsyncAPI (Event-Driven APIs) . . . 74
40.3.5 gRPC / Protocol Buffers . . . 74

40.4 Abstract vs Concrete API Contracts ... 74
40.4.1 Abstract Contracts (Schema-Only) ... 74
40.4.2 Concrete Contracts (Full API Specification) ... 74
40.4.3 The Relationship ... 74
40.5 Relational Data Contract Standards ... 74
40.5.1 Data Contract (datacontract.com) ... 74
40.5.2 ODCS (Open Data Contract Standard — PayPal) ... 74
40.5.3 Comparison: datacontract.com vs ODCS ... 74
40.6 How UTL-X Relates to API and Data Contracts ... 74
40.6.1 UTL-X as the Transformation Layer ... 74
40.6.2 Schema Formats in UTL-X ... 74
40.6.3 Future: Data Contract Integration ... 74
40.6.4 Future: Contract-Driven Transformation ... 74
41 Formats Not Yet Covered (and Why) ... 75
41.1 Formats on the Horizon ... 75
41.2 TOML (Tom's Obvious Minimal Language) ... 75
41.3 JSON-LD (JSON for Linked Data) ... 75
41.4 HL7 FHIR (Fast Healthcare Interoperability Resources) ... 75
41.5 EDI / EDIFACT (Electronic Data Interchange) ... 75
41.6 HL7 v2 (Health Level Seven Version 2) ... 75
41.7 Parquet / Arrow (Columnar Data) ... 75
41.8 MessagePack / CBOR (Binary JSON) ... 75
41.9 GraphQL ... 75
41.10 Apache Thrift ... 75
41.11 FlatBuffers ... 75
41.12 Summary: Format Priority Matrix ... 75
42 Competitive Landscape: Mapping and Integration Tools ... 76
42.1 Command-Line Transformation Tools ... 76
42.1.1 jq — The JSON Swiss Knife ... 76
42.1.2 yq — YAML/JSON/XML/CSV/TOML Processor ... 76
42.1.3 JSONata — JSON Query and Transformation ... 76
42.1.4 XSLT — The XML Veteran ... 76
42.2 Enterprise Integration Platforms (iPaaS) ... 76
42.2.1 MuleSoft (Salesforce) ... 76
42.2.2 Tibco BusinessWorks (BWCE) ... 76
42.2.3 Boomi (formerly Dell Boomi) ... 76
42.2.4 Workato ... 76
42.2.5 Informatica / IICS ... 76
42.3 Hyperscaler Native Tools ... 76
42.3.1 Azure ... 76
42.3.2 AWS ... 76
42.3.3 GCP ... 76
42.3.4 Hyperscaler Comparison Table ... 76
42.4 ERP Vendors ... 76
42.4.1 SAP ... 76
42.4.2 Oracle ... 76

42.4.3 Microsoft Dynamics 365 . . . 76
42.4.4 Odoo . . . 76
42.4.5 Summary: ERP + UTL-X Integration Pattern . . . 76
42.5 Open Source Alternatives . . . 76
42.5.1 Apache Camel . . . 76
42.5.2 Smooks . . . 76
42.5.3 Ballerina . . . 76
42.6 Positioning Matrix . . . 76
43 Open-M: The Integration Controller (Teaser) . . . 77
43.1 What Is Open-M? . . . 77
43.2 How UTL-X Fits in Open-M . . . 77
43.3 Why Open-M + UTL-X? . . . 77
43.4 Open-M Architecture (Preview) . . . 77
43.5 MPPM: Message Processing Pipeline Model . . . 77
43.5.1 What Is MPPM? . . . 77
43.5.2 MPPM Stages . . . 77
43.5.3 MPPM Message Envelope . . . 77
43.5.4 How MPPM Maps to UTL-X . . . 77
43.5.5 Why MPPM Matters for UTL-X Users . . . 77
43.6 What This Means for UTL-X Users . . . 77
44 AI and UTL-X: The Future of Automated Transformation . . . 78
44.1 The Current State: AI-Assisted UTL-X Development . . . 78
44.1.1 What Works Today . . . 78
44.1.2 The Problem: Token Cost and Context . . . 78
44.2 The Vision: A UTL-X Fine-Tuned LLM . . . 78
44.2.1 What Fine-Tuning Would Enable . . . 78
44.2.2 Token Savings Calculation . . . 78
44.3 Training Data: What Would Be Needed . . . 78
44.3.1 Available Training Corpus . . . 78
44.3.2 Additional Training Data Needed . . . 78
44.4 Fine-Tuning Approaches . . . 78
44.4.1 Option A: LoRA Fine-Tune on Open Model . . . 78
44.4.2 Option B: Fine-Tune on Frontier Model . . . 78
44.4.3 Option C: RAG (Retrieval-Augmented Generation) — No Fine-Tuning . . . 78
44.4.4 Recommended Path . . . 78
44.5 Investment Estimate . . . 78
44.5.1 Development Costs . . . 78
44.5.2 Ongoing Costs . . . 78
44.5.3 Revenue Potential . . . 78
44.6 What an Investor Would Want to Know . . . 78
44.6.1 The Opportunity . . . 78
44.6.2 The Moat . . . 78
44.6.3 The Ask . . . 78
44.6.4 Why NOW . . . 78
44.7 Ethical Considerations . . . 78
45 Why Kotlin and GraalVM (and Why Not Other Languages) . . . 79

45.1 Why Kotlin Was the Right Choice ... 79
45.1.1 Kotlin's Strengths for UTL-X ... 79
45.1.2 Kotlin vs Alternatives Considered ... 79
45.2 Why GraalVM on Top of Kotlin ... 79
45.2.1 GraalVM Native Image ... 79
45.2.2 GraalVM for Cloud (UTLXe) ... 79
45.3 Why NOT a JavaScript/TypeScript Version ... 79
45.3.1 The Temptation ... 79
45.3.2 Why It Would Fail ... 79
45.4 Why NOT WebAssembly (WASM) ... 79
45.4.1 The Promise ... 79
45.4.2 Why It Doesn't Work (Today) ... 79
45.5 Why Multiple Language Implementations Lead to Deviation ... 79
45.5.1 The N-Implementation Problem ... 79
45.5.2 The Wrapper Pattern Avoids Deviation ... 79
46 Case Studies and Real-World Use Cases ... 81
46.1 Case Study 1: European E-Invoicing (Peppol / UBL) ... 81
46.2 Case Study 2: Healthcare HL7 FHIR Integration ... 81
46.3 Case Study 3: Financial Services — SWIFT ISO 20022 Migration ... 81
46.4 Case Study 4: Retail — Multi-Channel Order Processing ... 81
46.5 Case Study 5: IoT / Manufacturing — Sensor Data Normalization ... 81
46.6 Case Study 6: Government — Tax Filing Integration ... 81
46.7 Azure Marketplace: Getting Started Guide ... 81
46.8 Recipes: Common Transformation Patterns ... 81
47 Grammar Reference ... 83
47.1 UTLX 1.0 Grammar (EBNF) ... 83
47.2 Header Syntax ... 83
47.3 Output Options ... 83
47.4 Expression Syntax ... 83
47.5 Reserved Words ... 83
47.6 Identifier Rules ... 83
48 Appendices ... 84
48.1 Appendix A: Format Conversion Matrix ... 84
48.2 Appendix B: Error Messages Reference ... 84
48.3 Appendix C: Comparison Tables ... 84
48.4 Appendix D: Industry Standards Reference ... 84
48.5 Appendix E: Glossary ... 84
48.6 Appendix F: Version History ... 84
48.7 Appendix G: License ... 84
49 Standard Library Reference ... 86
49.1 How to Read This Reference ... 86
49.2 Core Functions (49) ... 86
49.3 String Functions (83) ... 86
49.4 Array Functions (67) ... 86
49.5 Math Functions (37) ... 86
49.6 Date/Time Functions (68) ... 86

49.7 Type Functions (27) 86
49.8 Object Functions (1) 86
49.9 Encoding Functions (30) 86
49.10 XML Functions (60) 86
49.11 Security Functions (16) 86
49.12 JSON Functions (6) 86
49.13 Binary Functions (47) 86
49.14 Utility Functions (27) 86
49.15 Financial Functions (16) 86
49.16 Geospatial Functions (8) 86
49.17 CSV Functions (12) 86
49.18 YAML Functions (22) 86

1 Preface

1.1 Why This Book

1.2 Who This Book Is For

1.3 How to Read This Book

1.4 Conventions Used

1.5 Online Resources

Part I

Foundation

Understanding the language, the data model, and the philosophy

2 Introduction

Every enterprise integration project faces the same fundamental problem: systems don't speak the same language. Your ERP sends XML. Your REST API expects JSON. Your legacy batch process reads CSV. Your DevOps pipeline is configured in YAML. And somewhere, an OData service from Microsoft Dynamics 365 is waiting for its own peculiar JSON dialect.

Traditionally, you'd learn a different tool for each format. XSLT for XML. Liquid templates for JSON. Custom Python scripts for CSV. And for YAML? Good luck — most tools pretend it doesn't exist.

UTL-X changes this. One language. All formats. Write your transformation logic once, and it works whether your input is XML, JSON, CSV, YAML, or OData.

2.1 The Data Transformation Problem

Consider a typical integration scenario. A Dutch wholesale distributor receives purchase orders from 12 European partners. Partner A sends XML via SOAP. Partner B posts JSON to a REST API. Partner C drops CSV files on an SFTP server. Partner D sends YAML webhooks from their Kubernetes-based platform.

Without a format-agnostic tool, this distributor needs:

- An XSLT stylesheet for Partner A's XML
- Custom C# code for Partner B's JSON
- A Python script for Partner C's CSV
- Yet another script for Partner D's YAML

Four languages. Four maintenance burdens. Four sets of bugs. And when a new partner appears with OData from SAP? A fifth.

With UTL-X, all four partners are handled by the same transformation language. The `.utlx` file doesn't care whether the input is XML or CSV — it operates on the *Universal Data Model* (UDM), an internal representation that normalizes all formats into one tree structure.

2.2 What Is UTL-X?

UTL-X — *Universal Transformation Language Extended* — is a format-agnostic, functional transformation language for data integration. It is:

- **Format-agnostic**: one transformation works with XML, JSON, CSV, YAML, and OData
- **Functional**: declarative, composable, testable — no mutable state, no side effects
- **Open source**: licensed under AGPL-3.0, free for any use (see Chapter 2)
- **Production-ready**: 86,000+ messages per second, schema validation, pipeline chaining
- **Batteries included**: 652 built-in functions across 18 categories

At its core, UTL-X answers a simple question: *"Given this input data, produce that output data."* Everything else — format parsing, type coercion, attribute handling, namespace resolution — is handled automatically.

2.3 The Swiss Army Knife Analogy

UTL-X is often compared to a Swiss Army knife for data. The analogy is apt:

- Each **blade** is a format: XML, JSON, CSV, YAML, OData

- The **handle** is the UDM — the common core that connects all blades
- The **tools** are the 652 standard library functions: string manipulation, date arithmetic, array operations, cryptographic hashing, and more
- You carry **one tool** instead of five — and it handles any situation

2.4 UTL-X vs The Alternatives

How does UTL-X compare to existing transformation tools?

Capability	XSLT	DataWeave	jq	JSONata	UTL-X
JSON native	No	Yes	Yes	Yes	Yes
XML native	Yes	Yes	No	No	Yes
CSV native	No	Yes	No	No	Yes
YAML native	No	Yes	No	No	Yes
OData native	No	No	No	No	Yes
Schema validation	XSD	Limited	No	No	7 formats
Open source	Standard	No	Yes	Yes	Yes (AGPL)
Production engine	Yes	Yes	No	No	Yes
Functions	50	200	50	30	**652**
Cloud Marketplace	No	MuleSoft	No	No	**Azure/GCP/AWS**

XSLT is the 25-year veteran — powerful for XML, but XML-only and verbose. DataWeave (MuleSoft) is the closest commercial competitor — format-agnostic and functional, but proprietary and expensive ($1,250+/month). jq is the beloved command-line JSON processor — fast and elegant, but JSON-only. JSONata is a compact JSON query language — clean syntax, but limited scope.

UTL-X combines the breadth of DataWeave with the openness of jq, adds production deployment capabilities, and extends to formats that no other tool covers natively.

2.5 Architecture Overview

UTL-X's architecture is built around one central idea: *all data is the same, regardless of format.*

The pipeline works in three stages:

1. **Parse**: The input — whether XML, JSON, CSV, YAML, or OData — is parsed into the Universal Data Model (UDM). The UDM is a tree of typed nodes: scalars (strings, numbers, booleans), objects (with properties and optionally XML attributes), and arrays.

2. **Transform**: The `.utlx` transformation operates on the UDM tree. It can access any property, iterate arrays, apply functions, and construct new structures. The transformation is format-agnostic — it doesn't know or care whether the data came from XML or JSON.

3. **Serialize**: The resulting UDM tree is serialized to the target format. XML gets elements and attributes. JSON gets objects and arrays. CSV gets rows and columns. Each serializer handles format-specific concerns (XML namespaces, JSON types, CSV delimiters) transparently.

This three-stage architecture is what makes format-agnosticism possible. The transformation logic never touches raw XML tags or JSON braces — it works with the *meaning* of the data, not its *syntax*.

2.6 The UTL-X Ecosystem

UTL-X is not a single tool — it's an ecosystem of three executables, each designed for a different stage of the development lifecycle:

Executable	Purpose	When to use
`utlx`	CLI — command-line tool	Development, scripting, CI/CD
`utlxd`	Daemon — IDE integration	VS Code extension, live preview
`utlxe`	Engine — production runtime	Cloud deployment, Azure Marketplace

All three share the same parser, interpreter, UDM, and standard library. A transformation written in VS Code (via `utlxd`) runs identically on the command line (via `utlx`) and in production (via `utlxe`). Chapter 5 explains each executable in detail.

2.7 A First Example

Let's see UTL-X in action. Given this XML order:

```
<Order id="ORD-001">
  <Customer>Alice Johnson</Customer>
  <Items>
    <Item sku="WIDGET-01" price="50.00" qty="2"/>
    <Item sku="GADGET-02" price="75.00" qty="1"/>
  </Items>
</Order>
```

We want to produce this JSON:

```
{
  "orderId": "ORD-001",
  "customer": "Alice Johnson",
  "total": 175.00
}
```

Here is the UTL-X transformation:

```
%utlx 1.0
input xml
output json
---
{
  orderId: $input.Order.@id,
  customer: $input.Order.Customer,
  total: sum(map($input.Order.Items.Item,
    (item) -> toNumber(item.@price) * toNumber(item.@qty)
```

```
  ))
}
```

Let's break it down:

- `%utlx 1.0` — declares this is a UTL-X version 1.0 transformation
- `input xml` — the input data is XML
- `output json` — the output should be JSON
- `---` — separates the header from the body
- `\$input.Order.\@id` — accesses the `id` attribute on the `Order` element
- `\$input.Order.Customer` — accesses the text content of `Customer`
- `map(...)` — iterates over each `Item`, computing price times quantity
- `sum(...)` — adds up all the line totals

Run it:

```
cat order.xml | utlx transform order-to-json.utlx
```

Output:

```
{
  "orderId": "ORD-001",
  "customer": "Alice Johnson",
  "total": 175.0
}
```

That's UTL-X. One transformation file. Input XML, output JSON. All the format details — XML attributes, element text, JSON types — handled automatically.

Now change `output json` to `output yaml`, and the same transformation produces:

```
orderId: ORD-001
customer: Alice Johnson
total: 175.0
```

Or `output csv`:

```
orderId,customer,total
ORD-001,Alice Johnson,175.0
```

The transformation logic didn't change. Only the output declaration. That is the power of format-agnosticism.

2.8 What's Ahead

This book is organized in eight parts:

- **Part I** (you are here) lays the foundation: installation, the three executables, the UDM, schema mapping, and format support

- **Part II** teaches the language: expressions, functions, pattern matching, validation, and pipelines
- **Part III** dives deep into each format: XML, JSON, CSV, YAML, OData, and schema formats
- **Part IV** covers real-world applications: enterprise integration, cloud deployment, SDKs, performance, and compliance
- **Part V** looks ahead: semantic validation, API contracts, AI-assisted generation, and why Kotlin was the right choice
- **Part VI** presents case studies from e-invoicing, healthcare, financial services, and more
- **Part VII** provides reference material: grammar, appendices, and glossary
- **Part VIII** is the complete standard library encyclopedia: all 652 functions with signatures and examples

Whether you're an integration developer replacing XSLT, a data engineer building multi-format pipelines, or an architect evaluating transformation technologies — this book will take you from first transformation to production deployment.

Let's get started.

3 Licensing: Open Source and Commercial

Before diving into the technical details of UTL-X, it's important to understand what you can and cannot do with it. Licensing matters — especially in enterprise environments where legal and compliance teams review every dependency.

The short version: UTL-X is free and open source for almost every use case. The only exception is embedding it inside a proprietary product you distribute — and even then, a commercial license is available.

3.1 UTL-X Is Open Source (AGPL-3.0)

UTL-X is licensed under the **GNU Affero General Public License v3.0** (AGPL-3.0). This is one of the strongest open-source licenses, designed to keep software free and open — even when deployed as a cloud service.

If you've worked with GPL-licensed software before, AGPL is similar with one important addition: the *network use* clause. Under regular GPL, sharing obligations only trigger when you *distribute* the software. Under AGPL, they also trigger when you make the software available over a *network* — for example, as a SaaS offering.

This matters for UTL-X because transformation engines are typically deployed as services, not distributed as installable software.

3.2 What You Can Do (Without Paying Anything)

Under the AGPL-3.0 license, you are free to:

- **Use UTL-X for any purpose** — personal, educational, or commercial
- **Run UTL-X internally** within your organization, on any number of servers
- **Modify the source code** to fit your needs
- **Deploy UTL-X as a service** within your organization
- **Write any number of `.utlx` transformations** — they are yours (more on this below)

There is no usage limit, no trial period, no feature restriction. The full UTL-X engine with all 652 standard library functions, all 11 format parsers, and all execution strategies is available under AGPL-3.0.

3.3 The Key AGPL Condition

If you **modify** UTL-X and make your modified version available to users over a network (as a web service, API, or cloud service), you must make your modified source code available to those users under the same AGPL-3.0 license.

This is the *quid pro quo* of AGPL: you benefit from the open-source community's work, and in return, your improvements flow back to the community.

Note the emphasis on *modified*. If you use UTL-X as-is (without changing the engine source code), there is nothing to share — the original source is already public on GitHub.

3.4 Your Transformations Are Yours

This is the most important clarification in this chapter:

The AGPL license covers the UTL-X software — the parser, interpreter, engine, and standard library. It does NOT cover the transformations you write or the data you process.

Your `.utlx` files are *input* to the program, not part of the program. They are your intellectual property. You can keep them proprietary, sell them, or do anything you want with them.

The analogy is precise: GCC (the GNU C compiler) is licensed under GPL. But the C programs you compile with GCC are *yours* — GCC's license does not "infect" your code. Similarly, UTL-X's AGPL license does not affect your `.utlx` transformation files.

This means:

- Your transformation logic is private
- Your schemas are private
- Your pipeline configurations are private
- Your test data is private
- Only the UTL-X engine itself is AGPL

3.5 Usage Scenarios

The following table covers every common scenario:

Scenario	License	Cost	Why
Personal use / learning	AGPL (free)	Free	Any use is permitted
Internal company use (unmodified)	AGPL (free)	Free	No distribution, no network clause
Internal company use (modified)	AGPL (free)	Free	Internal use is not distribution
Open-source project using UTL-X	AGPL (free)	Free	Your project must be AGPL-compatible
Consulting (writing .utlx for clients)	AGPL (free)	Free	.utlx files are data, not software
Azure Marketplace deployment	Commercial	$35–105/mo	Commercial license included
GCP / AWS Marketplace	Commercial	$44–131/mo	Commercial license included
Embedding in proprietary SaaS	Commercial	Contact	AGPL requires source sharing
OEM / white-label distribution	Commercial	Contact	Distributing modified AGPL code

The vast majority of users fall into the first five rows — all free, no obligations.

3.6 The Subprocess Exception

A technical but important detail: if your application communicates with UTL-X via **HTTP API**, **subprocess** (stdio-proto), or **Dapr sidecar**, there is no *linking* between your code and UTL-X. Your application's license is not affected by AGPL.

The C# and Go wrappers use subprocess communication (protobuf over stdin/stdout). They spawn UTL-X as a separate process. This is not linking — it's inter-process communication. Your C#, Go, or Python application remains under whatever license you choose.

This is the same principle that allows proprietary applications to call GPL-licensed command-line tools (like `grep` or `git`) without becoming GPL themselves.

3.7 The Commercial License

3.7.1 Why It Exists

Some organizations have policies that prohibit AGPL dependencies — regardless of how they're used. Others want to embed UTL-X inside a proprietary product without source-sharing obligations. The commercial license serves these cases.

3.7.2 Cloud Marketplace (Simplest Path)

When you deploy UTL-X from the Azure Marketplace (or GCP/AWS when available), a commercial license is included in the subscription. No AGPL obligations apply. The licensing is handled by Microsoft/Google/Amazon through the Marketplace billing.

- **Azure**: Starter $35/month, Professional $105/month
- **GCP**: Starter $44/month, Professional $131/month
- **AWS**: Starter $44/month, Professional $131/month

3.7.3 Direct Commercial License

For companies embedding UTL-X in proprietary products, OEM distribution, or organizations with AGPL-incompatible compliance policies, a direct commercial license is available. Pricing is per-deployment or per-organization, negotiated based on use case.

Contact: licensing@utlx-lang.org

3.8 Dual Licensing: How It Works

UTL-X uses a *dual licensing* model — the same source code is available under two licenses. You choose which one applies to your use:

License A: AGPL-3.0 (free)	License B: Commercial (paid)
Use, modify, deploy freely	No source-sharing obligation
Share modifications if network service	Embed in proprietary products
Your .utlx files are NOT affected	Azure/GCP/AWS Marketplace includes this
Community-driven improvements	Direct support available

Most users never need the commercial license. The AGPL is free, permissive for internal use, and doesn't touch your transformation files. The commercial license exists as a safety net for the minority of use cases where AGPL is a blocker.

3.9 Frequently Asked Questions

"Can I use UTL-X in my company for free?"

Yes. Internal use is always free under AGPL. You can run UTL-X on your servers, write any transformations, process any data. No payment needed. No source-sharing needed — internal use is not distribution.

"Do I need to open-source my .utlx files?"

No. Your transformation files are data, not part of the UTL-X software. AGPL does not apply to them. They are your intellectual property.

"Can I use UTL-X in my SaaS product?"

If you use UTL-X *unmodified*: yes, under AGPL — but you must make clear to users that UTL-X (AGPL) is used, and provide access to its source code. If you *modify* UTL-X: you must share your modifications under AGPL. If you want no obligations: buy a commercial license.

"What if I deploy from Azure Marketplace?"

Commercial license is included. No AGPL obligations. You pay $35–105/month through your Azure bill.

"Does AGPL affect my entire application?"

Only if you *link* or *embed* UTL-X directly in your application code (e.g., importing UTL-X as a Java library). If you communicate via HTTP, subprocess, or Dapr — your application is not affected. The Go and C# wrappers use subprocess communication, specifically to avoid this.

With licensing clarified, let's move on to what matters most: using UTL-X to transform data.

4 Transformation: The Cornerstone of Integration

4.1 Why Transformation Matters

4.2 The Three Pillars of Integration

4.3 Transformation in ESB Architecture

4.4 Transformation in Workflow/BPM

4.5 Why UTLXe Needs Messaging Capabilities

4.6 The Canonical Data Model Pattern

5 Getting Started

This chapter gets UTL-X running on your machine in under five minutes. By the end, you'll have installed the CLI, run your first transformation, and understood the structure of a `.utlx` file.

5.1 Installation

UTL-X is distributed as a native binary — a single file, no JVM required, instant startup. Choose the method that fits your platform.

5.1.1 macOS (Homebrew) — Recommended

```
brew tap grauwen/utlx
brew install utlx
```

Verify:

```
utlx --version
# UTL-X CLI v1.0.2
# Universal Transformation Language Extended
```

Homebrew installs the GraalVM native binary. Startup time is under 10 milliseconds — as fast as `grep` or `jq`.

To update later:

```
brew update && brew upgrade utlx
```

5.1.2 Windows (Chocolatey)

```
choco install utlx
```

Verify:

```
utlx --version
```

If Chocolatey is not available, download the binary directly (see below).

5.1.3 Linux / macOS / Windows — Direct Download

Native binaries are available from GitHub Releases for every platform:

```
# macOS (Apple Silicon)
curl -L https://github.com/grauwen/utl-x/releases/download/v1.0.2/utlx-macos-arm
64 -o utlx
chmod +x utlx
sudo mv utlx /usr/local/bin/

# Linux (x64)
curl -L https://github.com/grauwen/utl-x/releases/download/v1.0.2/utlx-linux-x64
-o utlx
```

```
chmod +x utlx
sudo mv utlx /usr/local/bin/

# Windows: download utlx-windows-x64.exe from
# https://github.com/grauwen/utl-x/releases/tag/v1.0.2
# and place in a directory on your PATH
```

5.1.4 Building from Source

If you prefer to build UTL-X yourself, or need the JVM version for development:

Prerequisites:

- JDK 17 or later (`java -version` to check)
- Git

Steps:

```
# Clone the repository
git clone https://github.com/grauwen/utl-x.git
cd utl-x

# Build the CLI JAR
./gradlew :modules:cli:jar

# Run using the wrapper script
./utlx --version
```

The wrapper script (`utlx`) automatically finds and runs the compiled JAR. On Windows, use `utlx.bat` (Command Prompt) or `utlx.ps1` (PowerShell).

Building the native binary (requires GraalVM):

```
# Install GraalVM (macOS)
brew install --cask graalvm/tap/graalvm-community-jdk22

# Build native image
./gradlew :modules:cli:nativeCompile

# Binary at: modules/cli/build/native/nativeCompile/utlx
```

The native binary starts in under 10ms and uses 40MB of memory, compared to 250ms startup and 150MB for the JVM JAR. For development, the JAR is fine. For scripting and CI/CD, use the native binary.

5.1.5 Native Binary vs JVM JAR

Aspect	Native binary (GraalVM)	JVM JAR
Startup time	< 10ms	250ms
Memory	40 MB	150 MB
File size	80 MB (single file)	38 MB JAR + JVM

Requires JVM?	No	Yes (JDK 17+)
Install method	brew, choco, download	Build from source
Best for	CLI use, scripting, CI/CD	Development, debugging

When in doubt, use the native binary. It's what `brew install utlx` gives you.

5.2 Your First Transformation

Create a file called `hello.utlx`:

```
%utlx 1.0
input json
output json
---
{
  greeting: concat("Hello, ", $input.name, "!"),
  reversed: reverse($input.name),
  length: length($input.name)
}
```

Run it:

```
echo '{"name": "World"}' | utlx transform hello.utlx
```

Output:

```
{
  "greeting": "Hello, World!",
  "reversed": "dlroW",
  "length": 5
}
```

Congratulations — you've just run your first UTL-X transformation.

5.3 The .utlx File Structure

Every UTL-X transformation file has the same structure:

```
┌──────────────────────────────────┐
│ %utlx 1.0                        │ ← Version declaration
│ input json                       │ ← Input format
│ output xml                       │ ← Output format (+ options)
│ ---                              │ ← Separator
│ {                                │
│   name: $input.customer.name,    │ ← Transformation body
│   total: sum($input.items.price)│
│ }                                │
└──────────────────────────────────┘
```

The **header** (above `---`) declares:

- `%utlx 1.0` — the UTL-X language version (currently always 1.0)
- `input <format>` — what format the input data is in (xml, json, csv, yaml, odata)
- `output <format>` — what format to produce, with optional settings

The **body** (below `---`) is the transformation expression. It's a single expression that produces the output. No `return` keyword — the entire body IS the result.

5.3.1 Input Format Options

```
input json                          // standard JSON
input xml                           // XML (auto-detects encoding)
input csv                           // CSV (comma-delimited, with headers)
input csv {delimiter: ";"}          // semicolon-delimited
input yaml                          // YAML
input odata                         // OData JSON
```

5.3.2 Output Format Options

```
output json                              // pretty-printed JSON
output json {writeAttributes: true}      // preserve XML attributes
output xml {encoding: "UTF-8"}           // XML with encoding declaration
output csv {delimiter: ";", headers: true, regionalFormat: "european"}
output yaml                              // YAML block style
```

5.3.3 Multi-Input

When your transformation needs data from multiple sources:

```
%utlx 1.0
input: orders xml, customers json
output json
---
{
  orderCount: count($orders.Order),
  customerName: $customers.name
}
```

Each input gets its own variable name (prefixed with `$`).

5.4 Trying It Out: Five Quick Examples

Here are five transformations you can run right now to see UTL-X in action:

1. XML to JSON — the most common integration task:

```
echo '<User><Name>Alice</Name><Age>30</Age></User>' | utlx --from xml --to json
```

2. JSON to YAML — configuration format conversion:

```
echo '{"server": "prod-01", "port": 8080, "debug": false}' | utlx --from json --
to yaml
```

3. Extract a field — like jq but for any format:

```
echo '{"users": [{"name": "Alice"}, {"name": "Bob"}]}' | utlx -e
'map($input.users, (u) -> u.name)'
```

4. CSV to JSON — data import:

```
echo 'name,age\nAlice,30\nBob,25' | utlx --from csv --to json
```

5. XML attribute access — the `@` syntax:

```
echo '<Product id="P-001" price="29.99">Widget</Product>' | utlx --from xml -e
'{id: $input.Product.@id, name: $input.Product, price:
toNumber($input.Product.@price)}'
```

Each of these runs in under 10 milliseconds on a native binary. That's the UTL-X developer experience: instant feedback, any format.

5.5 What's Next

You now have UTL-X installed and running. The next chapters cover:

- **Chapter 5**: UTL-X on the command line — expression mode, identity flip, shell scripting
- **Chapter 6**: The three executables — CLI, daemon, and production engine
- **Chapter 7**: The IDE — VS Code extension, live preview, UDM tree browser

6 UTL-X on the Command Line

UTL-X was designed to feel at home in a terminal. If you've used `jq` for JSON processing, UTL-X will feel familiar — but it works with XML, CSV, YAML, and OData too. This chapter covers the three CLI modes, shell scripting patterns, and when to use each.

6.1 Three Ways to Use the CLI

Mode	Syntax	Best for
Transform	`utlx transform script.utlx input.xml`	Complex transformations with a .utlx file
Expression (`-e`)	`echo data | utlx -e '.name'`	Quick one-liners, field extraction, piping
Identity (flip)	`cat data.xml | utlx`	Instant format conversion, no script needed

6.2 Expression Mode: The jq Experience

Expression mode (`-e`) is the most powerful way to use UTL-X interactively. Instead of writing a `.utlx` file, you pass the transformation as a command-line argument:

```
echo '{"name": "Alice", "age": 30}' | utlx -e '.name'
# "Alice"
```

The dot (`.`) is shorthand for `$input`. So `.name` means `$input.name`. This mirrors jq's syntax — deliberately.

6.2.1 Raw Output (-r)

By default, string output includes quotes. Use `-r` (raw) to strip them — essential for piping to other commands:

```
# With quotes (default)
echo '{"name": "Alice"}' | utlx -e '.name'
# "Alice"

# Raw (no quotes) — pipe-friendly
echo '{"name": "Alice"}' | utlx -e '.name' -r
# Alice

# Use in shell variables
NAME=$(echo '{"name": "Alice"}' | utlx -e '.name' -r)
echo "Hello, $NAME"
# Hello, Alice
```

6.2.2 Expressions with Any Format

Unlike jq, expression mode works with XML, CSV, and YAML too:

```
# Extract from XML
echo '<User><Name>Alice</Name></User>' | utlx --from xml -e '.User.Name' -r
```

```
# Alice

# Extract from XML attribute
echo '<Product id="P-001"/>' | utlx --from xml -e '.Product.@id' -r
# P-001

# Extract from YAML
echo 'server: prod-01\nport: 8080' | utlx --from yaml -e '.port' -r
# 8080

# Extract from CSV (first row)
echo 'name,age\nAlice,30\nBob,25' | utlx --from csv -e '.[0].name' -r
# Alice
```

6.2.3 Complex Expressions

Expressions can be as simple or complex as needed:

```
# Object construction
echo '{"first": "Alice", "last": "Johnson"}' | utlx -e '{fullName: concat(.first,
" ", .last)}'
# {"fullName": "Alice Johnson"}

# Array operations
echo '[3, 1, 4, 1, 5, 9]' | utlx -e 'sort($input)'
# [1, 1, 3, 4, 5, 9]

# Filter and map
echo '{"users": [{"name": "Alice", "active": true}, {"name": "Bob", "active":
false}]}' \
  | utlx -e 'map(filter(.users, (u) -> u.active), (u) -> u.name)'
# ["Alice"]

# Aggregation
echo '[{"amount": 10}, {"amount": 20}, {"amount": 30}]' | utlx -e
'sum(map($input, (x) -> x.amount))'
# 60
```

6.3 Identity Mode: The Flip

The simplest way to use UTL-X — pipe data in, get a different format out. No script, no expression, no arguments:

```
# XML → JSON (automatic)
cat order.xml | utlx
# {"Order": {"Customer": "Alice", "Total": 299.99}}

# JSON → XML (automatic)
echo '{"name": "Alice"}' | utlx
# <?xml version="1.0" encoding="UTF-8"?><name>Alice</name>
```

UTL-X auto-detects the input format and “flips” to the most useful output format:

Input detected as	Output format (default flip)
XML	JSON
JSON	XML
CSV	JSON
YAML	JSON

Override the output format with `--to`:

```
# XML → YAML (instead of default JSON)
cat order.xml | utlx --to yaml

# JSON → CSV
echo '[{"name":"Alice","age":30},{"name":"Bob","age":25}]' | utlx --to csv

# CSV → YAML
cat report.csv | utlx --to yaml
```

Override the input format with `--from` (when auto-detection fails):

```
# Force input as CSV (e.g., when piped from another command)
some-command | utlx --from csv --to json
```

6.4 Shell Scripting Patterns

6.4.1 Batch File Conversion

Convert all XML files in a directory to JSON:

```
for f in *.xml; do
  utlx --from xml --to json < "$f" > "${f%.xml}.json"
done
```

6.4.2 Pipeline Integration

UTL-X is pipe-friendly — combine it with standard Unix tools:

```
# Extract customer names from XML orders, sort, deduplicate
cat orders.xml | utlx --from xml -e 'map(.Orders.Order, (o) -> o.Customer)' \
  | utlx -e 'sort($input)' \
  | utlx -e 'unique($input)'

# Count items in a JSON array
curl -s https://api.example.com/users | utlx -e 'count($input)' -r

# Filter CSV rows and convert to JSON
cat report.csv | utlx --from csv -e 'filter($input, (row) -> toNumber(row.amount)
> 100)' --to json
```

6.4.3 Using UTL-X in CI/CD

Validate a configuration file in a GitHub Actions workflow:

```
- name: Validate config
  run: |
    cat config.yaml | utlx --from yaml -e 'if (.version != null && .database !=
null) "valid" else "invalid"' -r
```

Transform API responses in deployment scripts:

```
# Extract image tag from Kubernetes deployment
kubectl get deployment myapp -o json | utlx -e
'.spec.template.spec.containers[0].image' -r
```

6.4.4 Combining with jq

UTL-X and jq can coexist. Use jq for JSON-only tasks where its syntax is more concise, and UTL-X when you need XML, CSV, or YAML:

```
# jq for simple JSON field extraction (shorter syntax)
echo '{"name": "Alice"}' | jq -r '.name'

# UTL-X when input is XML (jq can't do this)
echo '<User><Name>Alice</Name></User>' | utlx --from xml -e '.User.Name' -r

# Combine: UTL-X converts XML to JSON, jq processes further
cat complex.xml | utlx --from xml --to json | jq '.Orders[] | select(.total >
100)'
```

6.5 The UTL-X jq Cheat Sheet

For developers coming from jq, here's how common patterns translate:

Task	jq	UTL-X
Extract field	`jq '.name'`	`utlx -e '.name'`
Raw output	`jq -r '.name'`	`utlx -e '.name' -r`
Array length	`jq '. | length'`	`utlx -e 'count($input)'`
Map array	`jq '[.[] | .name]'`	`utlx -e 'map($input, (x) -> x.name)'`
Filter array	`jq '[.[] | select(.active)]'`	`utlx -e 'filter($input, (x) -> x.active)'`
Sort	`jq 'sort_by(.name)'`	`utlx -e 'sortBy($input, (x) -> x.name)'`
Keys	`jq 'keys'`	`utlx -e 'keys($input)'`
Sum	`jq '[.[].price] | add'`	`utlx -e 'sum(map($input, (x) -> x.price))'`
Compact	`jq -c '.'`	`utlx -e '$input' --no-pretty`

The key difference: every UTL-X command above also works with `--from xml`, `--from csv`, or `--from yaml`. jq is JSON-only.

6.6 When to Use Which Mode

Situation	Use
Quick field extraction	`utlx -e '.field' -r`
Format conversion (no logic)	`cat data.xml | utlx` (identity)
Format conversion (explicit target)	`cat data.xml | utlx --to yaml`
Complex transformation with reuse	`utlx transform script.utlx input.xml`
Shell pipeline	`some-cmd | utlx -e '...' -r | next-cmd`
CI/CD validation	`utlx -e 'if (...) "ok" else "fail"' -r`
Production service	Not CLI — use `utlxe` (see Chapter 6)

The CLI (`utlx`) is for humans and scripts. The engine (`utlxe`) is for production services. They share the same language — a transformation that works in `-e` mode works identically in `utlxe`.

6.7 Performance Notes

The native binary (GraalVM) is critical for CLI usage:

- **Startup**: < 10ms (native) vs 250ms (JVM JAR)
- **For single invocations**: startup time dominates — native is 25x faster end-to-end
- **For shell loops**: `for f in *.xml; do utlx ...; done` — native processes 100 files/second, JVM 4 files/second (startup penalty per invocation)
- **For pipes**: `cat data | utlx -e '...'` — native feels instant, JVM has noticeable delay

If you're using UTL-X in scripts or CI/CD, always use the native binary. The JVM JAR is for development only (when you need debugging or hot-reload).

7 The Three Executables: utlx, utlxd, and utlxe

7.1 One Language, Three Runtimes

7.2 utlx — The CLI (Command Line Interface)

7.3 utlxd — The Daemon (IDE Integration)

7.4 utlxe — The Engine (Production Runtime)

7.5 Comparison Table

7.6 Architecture: Shared Core, Different Shells

7.7 When to Use Which

7.8 The Wrapper Pattern

8 The UTL-X IDE

8.1 IDE Layout and Panels

8.2 Design-Time vs Runtime

8.3 Schema-to-Schema Mapping (Design-Time First)

8.4 Scaffolding

8.5 The UDM Tree Browser

8.6 The Function Library

8.7 The %utlx Header Editor

9 Language Fundamentals

9.1 Scalar Types

9.2 Objects

9.3 Arrays

9.4 The $input Variable

9.5 Variable Binding (let)

9.6 Conditional Expressions

9.7 Comments

9.8 Type Annotations (Optional)

10 The Universal Data Model (UDM)

10.1 Why UDM Exists

10.2 UDM Types

10.3 How Formats Map to UDM

10.4 XML-Specific UDM Conventions

10.5 UDM Navigation

10.6 UDM and Type Coercion

11 Schema-to-Schema Mapping

11.1 Why Schema Mapping Matters

11.2 How Traditional Tools Do It (and Why It's Broken)

11.2.1 The XSLT Approach (Tibco BW, SAP CPI, Azure)

11.2.2 The JSON Problem: Why JSON→XML→XSLT→XML→JSON Is Wrong

11.2.3 The UTL-X Approach: Format-Agnostic Schema Mapping

11.3 Tier 1: Instance Data Mapping

11.3.1 Every Data Format Has an Accompanying Schema Format

11.3.2 Instance-to-Instance Mapping Examples

11.4 Tier 2: Schema-to-Schema (Metadata) Mapping

11.4.1 Why Metadata Mapping Is Powerful

11.4.2 All Possible Schema-to-Schema Combinations

11.4.3 Why This Matters in Practice

11.5 JSONATA: JSON Schema to JSON Schema Mapping

11.6 Nested Schemas: Include and Import

11.7 USDL: The Universal Schema Definition Language

12 USDL: Universal Schema Definition Language

12.1 Why USDL?

12.2 USDL Syntax

12.3 USDL Type System

12.4 USDL Constraints

12.5 USDL ↔ Other Schema Formats

12.6 Using USDL in Transformations

12.7 USDL as Documentation

13 Format Support Overview

13.1 Tier 1: Data Formats

13.2 Tier 2: Schema Formats

13.3 Format Detection

13.4 Format Options

13.5 Format Conversion Matrix

Part II

The Language

Mastering expressions, functions, patterns, and advanced features

14 Expressions and Operators

14.1 Arithmetic Operators

14.2 Comparison Operators

14.3 Logical Operators

14.4 String Operators

14.5 Object Construction

14.6 Array Construction

14.7 Pipe Operator (|>)

14.8 Nullish Coalescing and Safe Navigation

15 Functions and Lambdas

15.1 Lambda Expressions

15.2 User-Defined Functions

15.3 Higher-Order Functions

15.4 Function Composition

15.5 Closures

15.6 The Standard Library (Overview)

16 Standard Library

16.1 String Functions (83 functions)

16.2 Array Functions (67 functions)

16.3 Math Functions (37 functions)

16.4 Date/Time Functions (68 functions)

16.5 Type Functions (27 functions)

16.6 Encoding Functions (30 functions)

16.7 Security Functions (16 functions)

16.8 XML Functions (60 functions)

16.9 Other Categories

17 Security Library and Message Security

17.1 The UTL-X Security Library (stdlib-security)

17.2 Should Messages Be Encrypted in Transit?

17.2.1 Transport-Level Encryption (mTLS)

17.2.2 Message-Level Encryption (End-to-End)

17.2.3 Why Encrypt-Transform-Re-encrypt Is a Maintenance Burden

17.3 Digital Signatures and Integrity

17.3.1 JCS (JSON Canonicalization Scheme)

17.3.2 XML Digital Signatures (XMLDSig)

17.3.3 JWS (JSON Web Signature)

17.4 Future Security Functions (Vision)

17.4.1 Encryption

17.4.2 Signing

17.4.3 Key Management Integration

17.5 Large Files: CMIS and Object Storage References

17.5.1 The Problem with Large Payloads

17.5.2 The Claim Check Pattern

17.5.3 CMIS (Content Management Interoperability Services)

17.5.4 Avoid Base64 Embedding

18 Pattern Matching and Control Flow

18.1 Match Expressions

18.2 Try/Catch

18.3 Conditional Patterns

18.4 Real-World Patterns

19 Schema Validation

19.1 Why Validate?

19.2 Supported Schema Formats

19.3 Pre-Validation and Post-Validation

19.4 JSON Schema Validation

19.5 XSD Validation

19.6 Schema-Driven Transformations

20 Pipeline Chaining and Multi-Hop Transformations

20.1 What Is Pipeline Chaining?

20.2 Why Pipelines?

20.3 Single-Hop vs Multi-Hop

20.3.1 Single-Hop (Simple Transformation)

20.3.2 Multi-Hop (Pipeline)

20.4 Multi-Hop with Additional Inputs per Stage

20.4.1 The Problem

20.4.2 The Solution: Additional Inputs per Hop

20.4.3 When Is Multi-Hop with Additional Inputs Handy?

20.4.3.1 1. Data Enrichment Pipelines

20.4.3.2 2. Multi-System Integration (Hub-and-Spoke)

20.4.3.3 3. Validation with External Reference Data

20.4.3.4 4. Template-Based Output Generation

20.4.3.5 5. Configuration-Driven Transformations

20.5 Defining Pipelines

20.5.1 Pipeline via UTLXe HTTP API

20.5.2 Pipeline via CLI

20.6 Multi-Input Transformations (Single Step)

20.7 Pipeline Patterns

20.7.1 Enrichment Pipeline

20.7.2 Validation Sandwich

20.7.3 Fan-Out

20.7.4 Fan-In (Aggregation)

20.7.5 Conditional Routing

20.8 Performance Characteristics

20.9 Status: Multi-Hop Additional Inputs

Part III

Formats Deep Dive

Working with XML, JSON, CSV, YAML, OData, and schema formats

21 XML Transformations

21.1 XML in UTL-X

21.2 Reading XML

21.3 Writing XML

21.4 XML Attributes in JSON/YAML Output

21.5 Common XML Patterns

21.6 XML Encoding

21.7 XML Namespaces Deep Dive

22 XML Attribute Handling: Design Decisions

22.1 The Fundamental Problem

22.2 How UTL-X Handles Attributes Internally (UDM)

22.3 The _text Convention

22.4 The writeAttributes Option

22.5 What Gets Kept vs Dropped (Default)

22.6 Why Drop by Default? (DataWeave-Compatible)

22.7 JSON vs YAML Consistency

22.8 Industry Conventions Reference

22.9 FHIR XML: The value Attribute Convention

23 JSON Transformations

23.1 JSON in UTL-X

23.2 Reading JSON

23.3 Writing JSON

23.4 JSON-Specific Functions

23.5 Common JSON Patterns

23.6 JSON Lines (JSONL)

24 CSV Transformations

24.1 CSV in UTL-X

24.2 CSV Options

24.3 Reading CSV

24.4 Writing CSV

24.5 CSV-Specific Functions

24.6 Common CSV Patterns

24.7 Regional Formats

25 YAML Transformations

25.1 YAML in UTL-X

25.2 Reading YAML

25.3 Writing YAML

25.4 YAML Output Options

25.5 YAML-Specific Functions

25.6 Common YAML Patterns

25.7 YAML ↔ JSON Conversion

26 OData Transformations

26.1 What Is OData?

26.2 OData in UTL-X

26.3 Reading OData

26.4 Writing OData

26.5 OData-Specific Functions

26.6 Common OData Patterns

26.6.1 Dynamics 365 Integration

26.6.2 SharePoint Integration

26.6.3 SAP OData Integration

26.6.4 Power Platform

26.7 OData Schema (EDMX/CSDL)

26.8 OData ↔ Other Formats

27 Schema Formats

27.1 XSD (XML Schema Definition)

27.2 JSON Schema

27.3 Avro Schema

27.4 Protobuf (proto3)

27.5 OSCH (OData Schema / EDMX)

27.6 TSCH (Table Schema)

27.7 USDL (Universal Schema Definition Language)

28 XSD Design Patterns Deep Dive

28.1 The Eight XML Schema Design Patterns

XML Schema (XSD) can be structured in fundamentally different ways. Understanding these patterns is essential for working with enterprise XML — and for understanding why schema-to-schema mapping is challenging.

28.1.1 1. Russian Doll

28.1.2 2. Salami Slice

28.1.3 3. Venetian Blind

28.1.4 4. Garden of Eden

28.1.5 5. Swiss Army Knife (Hybrid)

28.1.6 6-8. Chameleon, Abstract, Adapter

28.2 Pattern Detection in UTL-X

28.3 Pattern Conversion in UTL-X

28.4 XSD Patterns in Cross-Format Mapping

29 Cross-Format Patterns

29.1 The Power of Format Agnosticism

29.2 XML ↔ JSON Conversion

29.3 CSV ↔ JSON Conversion

29.4 XML → CSV (Flattening)

29.5 JSON → XML (Structuring)

29.6 Multi-Format Pipelines

29.7 Data Normalization Patterns

Part IV

Real-World Applications

Enterprise integration, cloud deployment, and production patterns

30 Enterprise Integration

30.1 UTL-X in Integration Architecture

30.2 Use Case: European E-Invoicing (Peppol/UBL)

30.3 Use Case: Healthcare (HL7 FHIR)

30.4 Use Case: Financial Services (SWIFT/ISO 20022)

30.5 Use Case: Retail/E-Commerce

30.6 Message Broker Integration

30.7 Error Handling in Production

31 Engine Lifecycle: Design-Time, Init-Time, Runtime

31.1 The Three Phases

Every UTL-X transformation goes through three distinct phases. Understanding them is key to performance tuning, debugging, and choosing the right execution strategy.

31.1.1 Phase 1: Design-Time

31.1.2 Phase 2: Init-Time (Load / Compile)

31.1.3 Phase 3: Runtime (Execute)

31.2 Phase Diagram

31.3 How Strategies Affect Each Phase

31.4 Worker Threading Model

32 Cloud Deployment

32.1 UTLXe: The Production Engine

32.2 Azure Deployment

32.3 GCP Deployment

32.4 AWS Deployment

32.5 Docker and Kubernetes

32.6 Health and Monitoring

32.7 Hot Reload

33 SDKs and Wrapper Libraries

33.1 The Wrapper Pattern

33.2 Available SDKs

33.2.1 C# / .NET Wrapper (Built)

33.2.2 Go Wrapper (Built)

33.2.3 Python Wrapper (Planned)

33.2.4 Java / Kotlin Wrapper (Direct)

33.2.5 Node.js / TypeScript Wrapper (Planned)

33.3 Alternative: HTTP API (No SDK Needed)

33.4 Alternative: Dapr Sidecar (Cloud-Native)

33.5 SDK Comparison

34 Migration Guides

34.1 Migrating from XSLT

34.2 Migrating from DataWeave

34.3 Migrating from jq

34.4 Migrating from Custom Code

34.5 Migrating from Tibco BW / BWCE

35 Performance and Optimization

35.1 Execution Strategies

35.2 Performance Characteristics

35.3 Memory Optimization

35.4 GraalVM Native Image

35.5 Benchmarking

35.6 Production Tuning

36 Message Parsing, Memory, and Size Boundaries

36.1 How UTL-X Parses Messages

36.1.1 DOM-Style Parsing (What UTL-X Uses)

36.1.2 SAX-Style Parsing (Not Used by UTL-X)

36.1.3 StAX-Style Parsing (Not Used by UTL-X)

36.1.4 Summary: Parsing Approaches

36.2 Memory Expansion Factor

36.2.1 The Problem: Why 100KB of XML Becomes 5-10MB in Memory

36.2.2 Expansion Factors by Format

36.2.3 Real-World Examples

36.3 Practical Message Size Boundaries

36.3.1 Sweet Spot: 1 KB - 1 MB

36.3.2 Feasible: 1 MB - 10 MB

36.3.3 Not Recommended: 10 MB - 50 MB

36.3.4 Not Feasible: 50 MB+

36.3.5 Size Boundary Summary

36.4 Memory Sizing Guide for Production

36.4.1 Formula

36.4.2 The 75% Rule

36.5 Future: Streaming Support?

36.5.1 Could UTL-X Support Streaming?

36.5.2 Split-Before-Transform Pattern

36.6 Comparison: UTL-X vs Streaming Tools

37 Logging, Observability, and Compliance

37.1 Logging in UTL-X

37.1.1 What UTL-X Logs

37.1.2 Log Levels

37.1.3 Configuring Logging

37.2 When NOT to Log: Compliance and Data Protection

37.2.1 PCI DSS (Payment Card Industry Data Security Standard)

37.2.2 GDPR (General Data Protection Regulation)

37.2.3 HIPAA (Health Insurance Portability and Accountability Act)

37.2.4 NEN 7510 (Dutch Healthcare Information Security)

37.2.5 SOX (Sarbanes-Oxley Act — Financial)

37.3 Designing for Compliance

37.3.1 The Principle: Log Metadata, Not Data

37.3.2 Audit Logging Pattern

37.3.3 Masking in Error Messages

37.4 Prometheus Metrics (Safe by Design)

38 Quality Assurance and Testing

38.1 UTL-X's Layered Testing Strategy

38.2 Layer 1: Unit Tests (Kotlin)

38.3 Layer 2: Format Tests (Kotlin)

38.4 Layer 3: Module Tests (Kotlin)

38.5 Layer 4: Integration Tests (Kotlin)

38.6 Layer 5: Conformance Suite (Python)

38.7 Auto-Capture: Recording Tests from CLI Usage

38.8 UTLXe Engine Tests

38.9 Test-Driven Transformation Development

38.10 Continuous Integration

Part V

Future Outlook

Semantic validation, API contracts, and the road ahead

39 Semantic Validation: Beyond Syntax

39.1 Syntactical vs Semantic Validation

39.1.1 What UTL-X Does Today: Syntactical Validation

39.1.2 What UTL-X Does NOT Do: Semantic Validation

39.1.3 The Gap

39.2 Semantic Validation Languages

39.2.1 Schematron (XML world)

39.2.2 JSONTron

39.2.3 FEEL (Friendly Enough Expression Language)

39.2.4 Other Approaches

39.3 How Semantic Validation Could Work in UTL-X

39.3.1 Option 1: Built-in assertion syntax

39.3.2 Option 2: Schematron support for XML output

39.3.3 Option 3: FEEL integration for cross-format validation

39.3.4 Option 4: Validation DSL (UTL-X native)

39.3.5 Recommended Approach

40 API Contracts and Data Contracts

40.1 What Is an API Contract?

40.2 Two Dimensions of Contracts

40.2.1 Hierarchical Contracts (API-Centric)

40.2.2 Relational Contracts (Data-Centric)

40.2.3 Key Difference

40.3 Hierarchical API Contract Standards

40.3.1 OpenAPI (REST APIs)

40.3.2 RAML (REST APIs — MuleSoft)

40.3.3 SOAP / WSDL (XML Web Services)

40.3.4 AsyncAPI (Event-Driven APIs)

40.3.5 gRPC / Protocol Buffers

40.4 Abstract vs Concrete API Contracts

40.4.1 Abstract Contracts (Schema-Only)

40.4.2 Concrete Contracts (Full API Specification)

40.4.3 The Relationship

40.5 Relational Data Contract Standards

40.5.1 Data Contract (datacontract.com)

40.5.2 ODCS (Open Data Contract Standard — PayPal)

40.5.3 Comparison: datacontract.com vs ODCS

40.6 How UTL-X Relates to API and Data Contracts

40.6.1 UTL-X as the Transformation Layer

40.6.2 Schema Formats in UTL-X

40.6.3 Future: Data Contract Integration

40.6.4 Future: Contract-Driven Transformation

41 Formats Not Yet Covered (and Why)

41.1 Formats on the Horizon

UTL-X currently supports 5 data formats (Tier 1) and 6 schema formats (Tier 2). Several important formats are NOT yet supported. This chapter explains each, why they matter, and the path to support.

41.2 TOML (Tom's Obvious Minimal Language)

41.3 JSON-LD (JSON for Linked Data)

41.4 HL7 FHIR (Fast Healthcare Interoperability Resources)

41.5 EDI / EDIFACT (Electronic Data Interchange)

41.6 HL7 v2 (Health Level Seven Version 2)

41.7 Parquet / Arrow (Columnar Data)

41.8 MessagePack / CBOR (Binary JSON)

41.9 GraphQL

41.10 Apache Thrift

41.11 FlatBuffers

41.12 Summary: Format Priority Matrix

42 Competitive Landscape: Mapping and Integration Tools

42.1 Command-Line Transformation Tools

42.1.1 jq — The JSON Swiss Knife

42.1.2 yq — YAML/JSON/XML/CSV/TOML Processor

42.1.3 JSONata — JSON Query and Transformation

42.1.4 XSLT — The XML Veteran

42.2 Enterprise Integration Platforms (iPaaS)

42.2.1 MuleSoft (Salesforce)

42.2.2 Tibco BusinessWorks (BWCE)

42.2.3 Boomi (formerly Dell Boomi)

42.2.4 Workato

42.2.5 Informatica / IICS

42.3 Hyperscaler Native Tools

42.3.1 Azure

42.3.2 AWS

42.3.3 GCP

42.3.4 Hyperscaler Comparison Table

42.4 ERP Vendors

42.4.1 SAP

42.4.2 Oracle

42.4.3 Microsoft Dynamics 365

42.4.4 Odoo

42.4.5 Summary: ERP + UTL-X Integration Pattern

42.5 Open Source Alternatives

42.5.1 Apache Camel

42.5.2 Smooks

42.5.3 Ballerina

42.6 Positioning Matrix

43 Open-M: The Integration Controller (Teaser)

43.1 What Is Open-M?

43.2 How UTL-X Fits in Open-M

43.3 Why Open-M + UTL-X?

43.4 Open-M Architecture (Preview)

43.5 MPPM: Message Processing Pipeline Model

43.5.1 What Is MPPM?

43.5.2 MPPM Stages

43.5.3 MPPM Message Envelope

43.5.4 How MPPM Maps to UTL-X

43.5.5 Why MPPM Matters for UTL-X Users

43.6 What This Means for UTL-X Users

44 AI and UTL-X: The Future of Automated Transformation

44.1 The Current State: AI-Assisted UTL-X Development

44.1.1 What Works Today

44.1.2 The Problem: Token Cost and Context

44.2 The Vision: A UTL-X Fine-Tuned LLM

44.2.1 What Fine-Tuning Would Enable

44.2.2 Token Savings Calculation

44.3 Training Data: What Would Be Needed

44.3.1 Available Training Corpus

44.3.2 Additional Training Data Needed

44.4 Fine-Tuning Approaches

44.4.1 Option A: LoRA Fine-Tune on Open Model

44.4.2 Option B: Fine-Tune on Frontier Model

44.4.3 Option C: RAG (Retrieval-Augmented Generation) — No Fine-Tuning

44.4.4 Recommended Path

44.5 Investment Estimate

44.5.1 Development Costs

44.5.2 Ongoing Costs

44.5.3 Revenue Potential

44.6 What an Investor Would Want to Know

44.6.1 The Opportunity

44.6.2 The Moat

44.6.3 The Ask

44.6.4 Why NOW

44.7 Ethical Considerations

45 Why Kotlin and GraalVM (and Why Not Other Languages)

45.1 Why Kotlin Was the Right Choice

45.1.1 Kotlin's Strengths for UTL-X

45.1.2 Kotlin vs Alternatives Considered

45.2 Why GraalVM on Top of Kotlin

45.2.1 GraalVM Native Image

45.2.2 GraalVM for Cloud (UTLXe)

45.3 Why NOT a JavaScript/TypeScript Version

45.3.1 The Temptation

45.3.2 Why It Would Fail

45.4 Why NOT WebAssembly (WASM)

45.4.1 The Promise

45.4.2 Why It Doesn't Work (Today)

45.5 Why Multiple Language Implementations Lead to Deviation

45.5.1 The N-Implementation Problem

45.5.2 The Wrapper Pattern Avoids Deviation

Part VI

Case Studies

Real-world use cases, step-by-step walkthroughs, and recipes

46 Case Studies and Real-World Use Cases

46.1 Case Study 1: European E-Invoicing (Peppol / UBL)

46.2 Case Study 2: Healthcare HL7 FHIR Integration

46.3 Case Study 3: Financial Services — SWIFT ISO 20022 Migration

46.4 Case Study 4: Retail — Multi-Channel Order Processing

46.5 Case Study 5: IoT / Manufacturing — Sensor Data Normalization

46.6 Case Study 6: Government — Tax Filing Integration

46.7 Azure Marketplace: Getting Started Guide

46.8 Recipes: Common Transformation Patterns

Part VII

Reference

Grammar, appendices, and general reference

47 Grammar Reference

47.1 UTLX 1.0 Grammar (EBNF)

47.2 Header Syntax

47.3 Output Options

47.4 Expression Syntax

47.5 Reserved Words

47.6 Identifier Rules

48 Appendices

48.1 Appendix A: Format Conversion Matrix

48.2 Appendix B: Error Messages Reference

48.3 Appendix C: Comparison Tables

48.4 Appendix D: Industry Standards Reference

48.5 Appendix E: Glossary

48.6 Appendix F: Version History

48.7 Appendix G: License

Part VIII

Standard Library Encyclopedia

Complete reference for all 652 functions — organized by category with examples

49 Standard Library Reference

49.1 How to Read This Reference

49.2 Core Functions (49)

49.3 String Functions (83)

49.4 Array Functions (67)

49.5 Math Functions (37)

49.6 Date/Time Functions (68)

49.7 Type Functions (27)

49.8 Object Functions (1)

49.9 Encoding Functions (30)

49.10 XML Functions (60)

49.11 Security Functions (16)

49.12 JSON Functions (6)

49.13 Binary Functions (47)

49.14 Utility Functions (27)

49.15 Financial Functions (16)

49.16 Geospatial Functions (8)

49.17 CSV Functions (12)

49.18 YAML Functions (22)